Practice Guide 03_03: Testing Anomaly Detectors as an Adversary Would

Aprendizagem Aplicada à Segurança

Mário Antunes

October 30, 2026

Objective

Measure how robust the detectors of guides 03_01 and 03_02 are, one kind of anomaly at a time. Point anomalies (KDD99): compute the price of evading a Mahalanobis detector, run a constrained mimicry and a budgeted black-box search against **your own** models, poison a retrained detector, and test vetted retraining. Contextual anomalies (a daily series): hide an offset under a ramp. Collective anomalies (a plant): lie on a sensor while keeping the correlations. Slow drifts are met with CUSUM.

- **Lecture:** `slides/slides_03_03.pdf`; **demos:** `notebooks/02-anomaly/` 07 to 09.
- **Notebook to complete:** `guide_03_03.ipynb` (this folder). Cells marked `TODO` are yours; `check` cells tell you whether you are right. Helpers: `anomlib.py`, `deeplib.py`, `robustlib.py` in `notebooks/02-anomaly/`.
- **Data:** KDD Cup 99 (deduplicated), Example 1 and Example 2 (generated).
- **Time:** about 2 hours (suggested: A 5 min, B 15, C 20, D 25, E 20, F 10, G 15, H 10, questions 15). The notebook runs in under a minute.
- **Scope:** our own models, public or synthetic data, offline, in feature space. The point is the defender's: which features an alarm rests on, and which defences raise the attacker's cost.

Background in one page

Threat model. Goal (evasion at test time, poisoning at training time), capability (which features may change), knowledge (normal data, scores, the model), cost (queries, rounds, a slower attack).

Constraints. A connection must still do what the attack needs: protocol, service, flag,

error rates and content counts are **fixed**; duration, bytes and connection counts (volume and timing) are **mutable**. An evasion that changes a fixed feature is not the same attack.

Price of evasion (Example 1). With $C = \begin{pmatrix} 1 & 0.9 \\ 0.9 & 1 \end{pmatrix}$, passing at 1% false alarms means $(x_1^2 - 1.8x_1x_2 + x_2^2)/0.19 \leq \chi^2_{2,0.99} = 9.21$: for fixed x_2, a quadratic inequality in x_1.

Mimicry. Copy the mutable features of the normal row nearest in the fixed features. **Black-box search:** one query = one score; change one mutable feature at a time, keep the change if the score drops, stop below the threshold or when the budget ends.

Boiling frog. A detector retrained on recent accepted data can be moved by points placed just inside its boundary, round after round. **Vetting:** retrain only on points that a frozen, trusted reference model also accepts.

Collective lie. A linear residual $\mathbf{r} = A\mathbf{x}$ (PCA) is minimised over the shifts of the sensors the attacker owns by least squares; with enough owned sensors the residual vanishes. **Ramp.** An offset that grows over days has no edge, so detectors that react to change miss it.

CUSUM. $S_t = \max(0, S_{t-1} + (r_t - \mu)/\sigma - k)$, alarm when $S_t > h$: accumulates small persistent deviations.

Tasks

Part	What you do	Check
A	Victims and mutable features (given)	
B	Implement `max_x1(x2, t)`	boundary scores exactly t
C	Implement `mimicry`	fixed features unchanged; table
D	Implement `random_search` with a query budget	budget respected; table
E	Implement `poison_round`; plain vs vetted retraining (given)	plot
F	Implement `cusum`; per-point vs CUSUM on a drift (given)	small array; table
G	Implement `lie`: least-squares shifts of the owned sensors; alarm rate against sensors owned	shapes; table
H	Implement `ramp`; seasonal difference against a step and a 3-day ramp	small array; table

Questions

1. **(B1)** What does the attacker lose by staying below the threshold? Is a detector useless if it can be evaded?
2. **(C1)** How many caught attacks evade after mimicry? Which categories evade most, and why? What does this say about the features these detectors rely on?
3. **(D1)** How does evasion grow with the budget, and when every feature is free? What limits the attacker: the search or the constraints?
4. **(D2)** Why is a feature-space evasion rate an upper bound for an attack on real traffic?
5. **(E1)** After how many rounds is the target accepted without vetting? Why does vetting stop it, and what legitimate change would it also slow down?
6. **(F1)** Why does CUSUM detect a slow drift earlier than a per-point threshold with the same false-alarm rule? What do k and h trade off?
7. **(F2)** Which other attack of this guide would CUSUM-like monitoring help against?
8. **(G1, G2)** How many sensors must the attacker own for a 2-sd lie to stay under the threshold? What would make the lie impossible?
9. **(H1, H2)** Which offsets does the seasonal difference miss for a step and for a 3-day ramp? Why does daily retraining on accepted data make it worse?

Detector	Caught	Evaded (mimicry)	Evaded (search, 500 queries)	Evaded (all features free)
Mahalanobis				
PCA				

Optional challenges

- **Robust features:** fit Mahalanobis and PCA on the fixed features only (`Xtr[:, ~mut]`). What is the clean recall, and what is the mimicry evasion (notebook 09, part B)?
- **Ensemble:** combine the three victims with `rl.RankEnsemble` and measure clean AUPRC and mimicry evasion. Is the ensemble more robust than its best member?
- **MCD under poisoning:** replace the Mahalanobis detector of part E with `an.Mahalanobis(robust=True)`. Does the robust estimator resist the boiling frog? Explain with what MCD discards.

Reference solution

The solved notebook, with the answers to the questions and the numbers of a full run, is `solution/guide_03_03.ipynb`.